VYLNT Technical Report and Presentation Guide

Technical architecture measured results and engineering defense

Version 2.0
Evidence cutoff 7 November 2026
Prepared for hackathon judges and senior engineering interviewers

VYLNT is a working web security configuration observer with a browser extension, React dashboard, authenticated persistence, reproducible insights reports, a local EVM digest registry and GitHub security checks. Its LSTM release is qualified only for controlled script-integrity analysis, JS-002. The other seven categories use conservative evidence-based reporting while their new project-backed ML qualification remains unfinished.

This report follows one scan through collection, detection, analysis, storage, export, verification and CI. It also explains the developer path that turns controlled application tests into a qualified model. Source observations, measured experiments, calculated statistics and proposed improvements are distinguished throughout.

Contents

1 Executive Summary and High Level Architecture 3

2 Complete Technology Stack and Design Decisions 6

3 Core Features and Component Breakdown 9

4 Machine Learning Pipeline Algorithms and Mathematical Rigor 15

5 Empirical Proof of ML Performance and Model Validation 19

6 Deployment Scalability and Production Readiness 23

7 Judge and Interview Defense Cheat Sheet 25

Appendix A Presentation and Live Demo Guide 27

Appendix B Evidence Register and References 30

Appendix C Follow One Report From Browser to GitHub 32

Appendix D Closed-Book Interview Drill 37

How to use this document

Read Sections 1 to 3 for the architecture. Study Appendix C alongside the dashboard to trace a report end to end. Then use Sections 4 and 5 to explain the model and its evidence, Section 6 for operational trade-offs, and Section 7 for the hardest defenses. Appendix A is the live demo script. Appendix D is a closed-book question bank. Appendix B maps claims to repository evidence.

Evidence labels used in the report are: IMPLEMENTED for source-backed behavior; RECORDED for previously executed checks; CALCULATED for arithmetic derived from published counts; OPEN for unfinished acceptance; and PROPOSED for future design. Recorded tests are not represented as newly rerun for this documentation task. The repository working tree includes development changes, so the Git commit alone does not identify every described increment.

1 Executive Summary and High Level Architecture

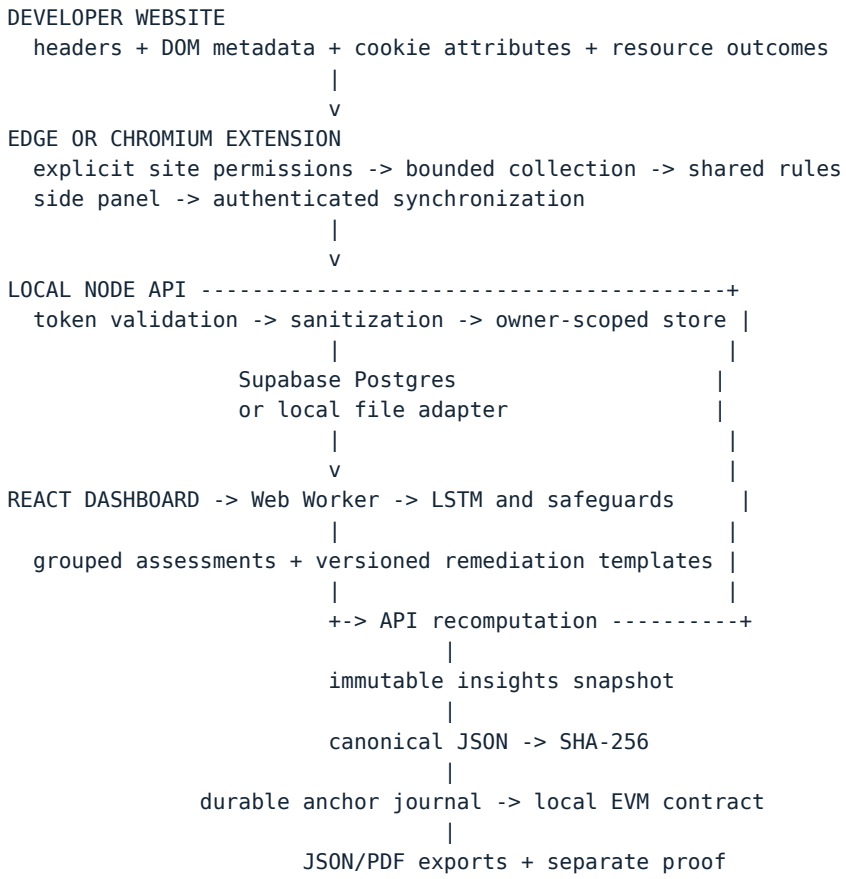
1.1 Problem and intended users

Browser-visible security configuration is difficult for a developing team to interpret consistently. A missing attribute may warrant a change, may be intentional, or may need application context unavailable to an external observer. A raw scanner can also repeat the same warning for dozens of resources, producing long reports without clarifying what to inspect first.

VYLNT addresses this workflow for students, hackathon teams, independent developers and small application teams. It collects bounded metadata, applies reproducible rules, groups related observations and presents evidence with cautious guidance. It helps developers inspect the same release repeatedly and preserve a report's identity. It is not a substitute for penetration testing, source-to-sink analysis, authorization review or a complete application-security program.

The intended impact is earlier discovery of configuration problems, lower manual triage effort and clearer evidence in development discussions. Potential business value is reduced review friction and repeatable release checks. These are product hypotheses: no controlled study of remediation time, adoption, prevented incidents, revenue or return on investment has been performed.

1.2 Architecture and trust boundaries



Four boundaries matter. Website input is untrusted. The extension receives only the browser permissions granted to it. The authenticated API independently derives trusted reports instead of accepting arbitrary client verdicts. The blockchain stores a digest of report content, while assessment quality remains the responsibility of the collection, rule and ML components.

Offline training is a separate developer-owned pipeline. Ordinary users do not prepare training datasets or start background learning. The local-project companion is another ingress path: an explicitly registered repository is built at a selected committed revision and scanned in a temporary process environment. It is not automatically attached to an older extension scan by URL matching.

1.3 End to end data flow

1. A user opens a permitted HTTP(S) website and requests a scan in the extension side panel. Browser-internal and inaccessible pages cannot be treated as successful scans.
2. The collector obtains the available response policies, script/resource metadata and cookie attributes. Cookie values, credentials, request bodies and page text are excluded from stored evidence. URL credentials, fragments and query strings are stripped.
3. Shared pure TypeScript rules derive findings, severity, confidence and a heuristic configuration score. Missing collection is recorded separately from an absence of findings.

4. The extension sends the sanitized observation to the authenticated local API. The server validates the submission, associates ownership and persists a consistent derived scan through the configured storage adapter.
5. The dashboard loads the saved scan and automatically starts a Web Worker. The current controlled release assesses eligible JS-002 contexts; the remaining categories retain evidence-based assessments.
6. Findings become grouped issues with their original references, facts, missing context and template-based guidance. The API recomputes the analysis when finalizing a saved version.
7. The immutable combined snapshot receives a canonical SHA-256 digest. Anchoring is queued independently of rendering and downloading. The report remains usable during chain outages.
8. The user downloads the full-data PDF or the separate insights PDF/JSON. Verification of an uploaded JSON report recomputes the trusted content and, when available, checks its live local anchor.

1.4 Current completion boundary

Capability	Current status	Defensible claim
Eight shared configuration checks	Implemented	Repeatable metadata-based findings, not proof of exploitation
Extension and dashboard workflow	Implemented with recorded browser checks	Localhost default and explicit additional-site permissions
JS-002 controlled LSTM	Qualified within recorded scope	One approved category on controlled script/CSP scenarios
Other seven ML categories	Open	326 of 630 eligible development examples in dataset 056, with 0 of 7 categories ready to fit
Frozen report identity	Implemented	API-verified immutable content versions and reproducible exports
Blockchain	Completed local demonstration scope	Real EVM transactions on a disposable local chain
GitHub integration	Two implemented workflows	Fresh-commit scanning and a separate uploaded-report demo check
Production hosting and fleet scale	Deferred	No deployment, public-chain or production-throughput claim

2 Complete Technology Stack and Design Decisions

2.1 Stack inventory

The versions below are the dependency declarations in the inspected package manifest, rather than a claim that a caret range is an immutable resolved version. The lockfile is the installation authority. The qualified training artifact separately records its historical Python library versions.

Layer	Tools and declared versions	Responsibility
Frontend	React ^19.3.1, React DOM ^19.3.1, TypeScript ^5.9.0, Vite ^6.4.0	Dashboard, report controls and browser build
Browser integration	Manifest V3, Chrome extension APIs, minimum Chromium version 120	Service worker, side panel, permissions and collection
Shared validation	TypeScript and Zod ^4.6.7	Input contracts and pure rule reuse
API	Node.js, Express ^5.3.0, Helmet ^8.3.0	Authenticated REST endpoints and response hardening
Identity and database	Supabase JS ^2.109.0, Supabase Auth, PostgreSQL/RLS	Account identity, ownership and persistent queryable scans
Local persistence	JSON file adapter and ignored journals	Local development storage and durable anchor jobs
Browser automation	Playwright ^1.64.1	Exact-commit collection, browser acceptance and report rendering
PDF	Chromium rendering, pdf-lib ^1.17.1, fontkit ^1.1.1	Application scan/insights export and PDF finalization
Blockchain	Hardhat 3.21.0, Solidity 0.8.38, ethers 6.17.0	Local EVM and digest registry
Rate limiting	express-rate-limit ^8.7.1, rate-limit-redis ^6.0.1, Redis client ^6.3.0	Local limits and optional shared limiter backend
Quality and CI	Vitest ^4.1.11, axe-core/Playwright, GitHub Actions, PGlite ^0.5.10	Unit/browser/accessibility/SQL checks and security gates
Qualified training record	Python 3.12.6, PyTorch 2.7.1+cpu, NumPy 1.26.4, scikit-learn 1.5.2	Offline fitting, comparator, calibration and numerical parity

The desktop's default Node 20.18.0 appears in historical measurements. Hardhat 3 tooling and native SQLite developer checks use the available Node 24 runtime. These are different

executable environments; a demo should verify the runtime selected by each wrapper rather than assuming one global Node version. [R4]

2.2 Frontend and collection decisions

React supports stateful scan history, asynchronous saves and cancellable analysis while reusing the team's TypeScript models. Vite provides a direct browser build. This implementation does not need Next.js server rendering because report analysis and the current dashboard are client-side and the API is separately served. This is an engineering rationale inferred from the implementation, not a documented comparative performance experiment.

A side panel keeps collection and scan feedback beside the inspected website. Manifest V3 service workers can stop and restart, so durable state cannot rely solely on global JavaScript variables. Browser event metadata, explicit coverage and bounded caches support this lifecycle. A persistent background page would simplify state management but conflicts with the chosen extension platform. Chrome's service-worker lifecycle documentation motivates restart-aware design. [R3]

TypeScript and pure shared rules reduce drift between the extension, API and CLI. Zod validates data crossing runtime boundaries; TypeScript alone cannot validate an incoming JSON object. Separate implementations in Python and JavaScript are retained only where needed for model training/runtime parity and tested against one another.

2.3 API and persistence decisions

Express is sufficient for the current authenticated REST workflow and keeps the API in the same language as the shared rule engine. A Python/FastAPI service could serve PyTorch directly, but it would add a running inference service, network latency and another deployment surface. The deployed model is small enough to execute numerically in the browser and server JavaScript runtimes.

Supabase combines identity with PostgreSQL ownership policies. Its relational model supports scan history, website summaries, immutable report versions and source-scan linkage. Row-level security provides a database enforcement boundary in addition to owner checks in application code. MongoDB or a custom authentication stack are viable, but would require different query, ownership and operational work. No measured database superiority is claimed. [R2]

The local file adapter supports development without a hosted database connection. Its whole-file reads and rewrites are suitable only within their tested limits; it is not a horizontally scalable database. Anchor jobs use a separate local journal because immutable report content must not change when a transaction's operational status changes. This also permits local blockchain demonstrations with either report-storage adapter.

2.4 ML and reporting decisions

The eight-unit LSTM is a compact sequence model over structured evidence. It can combine bounded context steps with the target observation and is straightforward to export as numeric weights. A logistic comparator uses the same ordered input information to test whether neural recurrence adds useful value. An LSTM was selected as the project architecture, but the recorded comparator policy does not establish universal superiority over logistic regression, decision trees or gradient boosting. These alternatives need fair future experiments on suitable native data.

A Web Worker keeps analysis away from the main UI thread. A numeric TypeScript runtime avoids an ONNX/TensorFlow.js runtime dependency for this small network. The trade-off is maintaining and testing the recurrent implementation and artifact parser. Versioned local templates supply remediation text; no external language model, AI provider credential or generated free-text security judgment is required.

2.5 Blockchain and infrastructure decisions

Hardhat provides an inspectable local EVM without real funds or browser wallets. Solidity encodes writer authorization and immutable first-write digest semantics; ethers connects the API worker to the contract. A standard database hash would already support local consistency checks. The chain adds a separate transaction/block history for the demonstration, but a chain controlled by the same operator does not provide independent public attestation.

GitHub Actions supplies a familiar review-time automation boundary. The fresh-commit workflow can build and scan submitted code on a clean runner. The lighter uploaded-report workflow is easier to copy into a demonstration repository but has weaker evidence binding. Docker, public deployment, wallet integration and a GitHub App are outside the selected scope. No recurring cloud price, production budget or scale claim is made.

3 Core Features and Component Breakdown

3.1 Extension lifecycle and collection coverage

The extension accepts the current tab and the permissions available for its origin. Its host permissions default to localhost/127.0.0.1, with optional HTTP(S) origins requested explicitly. Required APIs include storage, tabs, sidePanel, scripting, webRequest, webNavigation and cookies. Broad optional permission capability is not the same as an automatically granted permission for every website.

Collection produces an observation containing a sanitized page URL and time, allowlisted response policies, script attributes, resource outcomes, DOM coverage and cookie attributes. Missing headers or cookies are represented as missing collection rather than fabricated empty results. Request identities are tracked independently so two URLs with different private queries cannot overwrite one another merely because the stored URL is sanitized to the same path.

Failures include denied site access, unsupported browser URLs, missing response events, navigations during collection and collection caps. The side panel should explain the unavailable coverage or blocked scan. An authenticated synchronization error must remain distinguishable from successful local collection. A successful sign-in by itself does not prove a scan was uploaded.

3.2 Shared rule engine

Rule	Observation checked	Interpretation boundary
HDR-001	Missing/disabled HSTS on HTTPS	Applicability, valid TLS and policy details matter
HDR-002	Missing/insufficient framing policy	Intentional embedding can require different configuration
HDR-003	Missing/broad script policy	Dynamic trust and application compatibility need context
CK-001	Potential session cookie without Secure	Lifetime/name are heuristics, not proof of authentication purpose
CK-002	Potential session cookie without HttpOnly	JavaScript necessity must be established separately
CK-003	Cross-site cookie configuration requiring review	Necessary cross-site flows and CSRF protection are distinct facts
NET-001	HTTP resource referenced by an HTTPS document	A reference is not proof of successful unprotected execution
JS-002	Third-party script without integrity metadata	CSP restrictions, loading and delivery style affect relevance

Rules are pure functions over sanitized observations and produce deterministic findings. Original severity, confidence and the heuristic score remain rule outputs. An ML relevance score neither replaces severity nor estimates a probability of exploitation. DOM XSS, server-side injection, business logic and source-to-sink vulnerability detection are outside these eight checks.

3.3 Authentication storage and history

API ingress validates the account token and associates records with the authenticated owner. Local storage and Supabase implementations share application contracts. PostgreSQL RLS and owner-scoped functions protect database records; client-supplied owner identifiers are not trusted ownership assertions. Report finalization also checks source linkage and deterministic analysis derivation.

History exposes bounded pages and summary queries rather than requiring every full report for every view. Website summaries, comparisons and deletion support repeated review. Raw-scan deletion removes linked stored versions according to the implementation's ownership and cascade rules. A previously broadcast blockchain digest cannot be erased by deleting the corresponding database row.

The API generally limits JSON submissions to 512 KiB; the frozen combined insights envelope has a separate 1 MiB limit. Upload verification uses a specifically larger parser allowance and still validates the typed report envelope. Oversized reports are rejected explicitly rather than

dropping issues to force a fit. Size limits are not a statement about maximum generated PDF bytes.

3.4 Automatic analysis and grouped reports

```
saved scan -> worker startup -> release/feature checks
           -> per-variant inference -> safeguards
           -> per-category conservative aggregation
           -> evidence and template narrative -> automatic finalization
```

The worker calls the shared evidence-analysis entry point. It verifies the release identity, model hash, approved category policy and schema/ruleset compatibility. Only JS-002 reaches the active controlled-category model decision path. Other categories retain evidence-based assessments; for example, supported mixed-HTTP-resource evidence can recommend a change. Unsupported or insufficient context requires review.

Equivalent numeric contexts are cached so repeated observations do not require independent inference. Findings are grouped by category and retain their evidence variants and all finding IDs. In a group, any change-warranted member takes precedence; otherwise any uncertain member keeps the group under review. A group becomes contextual only when its members support that assessment. High/critical observations remain visible for review and repetition does not raise severity.

The assessment states are Change warranted, Needs review and Contextual. Missing data is not converted into a harmless verdict. Explanations identify observed facts, uncertainty, recommended configuration changes, compatibility considerations and verification steps. The prose comes from a versioned remediation catalog, not LSTM-generated language. Grouping 71 equivalent script observations into one issue is deterministic engineering behavior, not evidence of learned accuracy.

3.5 Frozen versions exports and uploaded verification

The dashboard automatically requests finalization after analysis completes. It submits an analysis identity rather than trusting arbitrary browser narrative. The authenticated API recomputes analysis from the saved scan, checks limits and freezes the content. Ordinary reports use the legacy frozen envelope; project-backed reports use a new envelope containing the original scan, original scan hash, analysis and signed project evidence.

The project envelope explicitly records that the new project-backed assessment is not enabled. Retaining provenance does not mean the unfinished evidence-v4 model is deployed. Archived models and parsers are preserved so saved content can still be re-derived and verified after a newer release exists. The same canonical content reuses its content identity; evidence/model/policy changes produce a different version.

There are two intentionally different user outputs. Print/PDF contains the full raw findings, severity and evidence without generated actions or ML. Download insights PDF/JSON contains the grouped assessment and reviewed guidance. Frozen insight content receives the report digest; blockchain operational metadata is downloaded separately so a new receipt does not silently alter the content hash.

The upload verifier accepts supported JSON structures, not an arbitrary edited PDF. It checks content integrity and trusted derivation/source linkage, then optionally a live anchor. Formatting-only JSON changes can preserve canonical identity. A modified PDF file is not automatically detected by a digest defined over canonical report JSON; a PDF-byte signature/hash would be a separate feature.

3.6 Local blockchain lifecycle

The non-upgradeable VylntReportAnchor contract uses Solidity 0.8.38. Its immutable writer is the deployer; its immutable nonzero deployment-instance identifier distinguishes environments. A nonzero bytes32 digest is stored at its first block number and an Anchored event is emitted. Repeated submissions do not overwrite the first record. Only report digests and transaction metadata go on-chain.

```
saved and verified report -> queued
  -> signed transaction durably recorded -> submitted
  -> successful canonical receipt and lookup -> confirmed
  -> transient error -> bounded backoff -> retry
  -> five failed attempts -> failed -> authenticated retry
```

A serialized transaction worker controls nonce allocation. Signed bytes are saved before broadcast so recovery can reconcile a lost response and resend the same transaction instead of immediately allocating a new nonce. Idempotency is scoped to owner, report digest and deployment instance. A successful report save is independent of successful queue persistence; repeated finalization can repair the missing job and the user receives a separate warning.

Live verification recomputes canonical content/source linkage and checks deployment identity, bytecode, receipt success, matching event, canonical block identity and contract lookup. Cached confirmed metadata alone does not establish a live match. Distinct outcomes include matching, content mismatch, not anchored and verification unavailable. One mined confirmation is sufficient for this local demo, not a public-network finality policy.

The local chain is disposable. Restart/reset or redeployment can invalidate current verification while historical receipts remain in the journal. It is one shared chain and contract, not a separate blockchain per user. Anonymous account grouping in the demo ledger comes from the API journal, not account identifiers stored by the contract. Public/demo ledger metadata has its own privacy exposure even when names and report contents are absent.

3.7 GitHub and local push workflows

The stronger fresh-commit gate checks a committed npm project using its versioned gate configuration, builds and starts a temporary checkout, scans declared loopback pages, and evaluates a pure blocking policy. High/critical findings at high confidence block. Other findings warn. Missing required collection, build failure, startup failure, timeouts or incompatible input fail closed. CLI exit codes are 0 for pass, 1 for blocking findings and 2 for execution/configuration failure.

The local pre-push hook checks distinct proposed branch tips; it excludes uncommitted fixes and ignores branch deletions/tags under the documented scope. It preserves existing hook configuration and refuses silent overwrites. Hooks can be bypassed. GitHub checks run after

branch pushes; blocking a merge requires separately configured repository protection. The user removed that enforcement requirement from the project's closure scope, so the project must not claim universally protected branches.

The dashboard-generated uploaded-report workflow evaluates committed VYLNT JSON. It can fail when the uploaded findings contain a blocker and pass after a new report no longer contains that blocker. It does not establish freshness, website-to-commit binding, a trusted source signature or live blockchain verification. Editing a report is not proof that website code was fixed. This weaker workflow must be identified explicitly during a demonstration.

The inspected repository workflow uses Node 24, read-only contents permissions, push/pull_request events, a stable VYLNT security gate job name, clean installation and artifacts retained seven days. It executes no pull_request_target application code. Actions are referenced by major tags in the inspected workflow; pinning all external actions by immutable SHA is additional supply-chain hardening.

3.8 Optional local project companion

Users register an explicit repository once through a one-time pairing code and a local CLI claim. The code expires after five minutes. They then choose a committed revision in the dashboard and start/cancel a run. The API builds and starts an isolated temporary checkout, collects a new scan, signs evidence and exposes progress. A local Git commit is required; a GitHub push is not required to inspect that commit.

The companion can retain supported Express source metadata and controlled browser facts unavailable to a normal extension scan. It executes trusted project code under the same OS account; a temporary process/check-out environment is not an operating-system sandbox. Ordinary scan upload endpoints cannot self-declare verified project evidence or choose arbitrary filesystem paths.

Installation-held Ed25519 keys bind evidence to run, commit, configuration, adapter identity and original scan hash. A pseudonymous owner tag protects report authorization. Archived public keys support old versions. The local OS user or malicious code with equivalent access remains inside this trust boundary. The wider native evidence adapters and seven-category model are unfinished; the local-project feature is optional for ordinary extension/dashboard use.

3.9 Worked example from observation to decision

Suppose a controlled HTTPS page references `http://example.test/widget.js`. The scanner records the sanitized reference, resource type and any observed transfer outcome. The rule engine emits NET-001 at high severity and high confidence for this script reference. An image reference would receive medium severity under the same rule. That confidence describes the observed configuration. It does not establish that the browser executed an insecure script.

The dashboard keeps the original finding, groups related NET-001 observations and explains the evidence. If execution or transfer context is missing, say precisely what is unknown. If the exact-commit gate finds the high/high script observation with complete required scan coverage, its rule predicate blocks the commit under test. If the reference is changed to

HTTPS and the gate rescans the new committed revision successfully, that particular observation can disappear. Any other blockers still apply.

The original and fixed scans have distinct IDs and contents. Each saved insights version gets its own canonical digest and optional anchor. The earlier digest continues to identify the earlier report. It provides an audit trail for what VYLNT recorded before the fix.

4 Machine Learning Pipeline Algorithms and Mathematical Rigor

4.1 Problem definition and labels

The learned task is supervised binary relevance classification, with an abstaining decision policy. For a supported observation, y=1 denotes change warranted and y=0 denotes contextual. Needs review is an uncertainty state, not a third class fitted by the current binary loss. Conflicting and insufficient-context examples are retained for auditing but excluded from eligible binary fitting.

This is neither regression nor NLP/CV. It does not analyze page prose, cookie values or raw JavaScript. It assesses structured evidence surrounding existing rule findings. A high relevance score is conditional on the supported category, feature contract and tested application scope; it is not an exploit probability or a statement that the website is secure.

4.2 Feature engineering and sequence representation

The active evidence-v3 release has 46 features per step, at most 74 steps and eight hidden units. Its context includes collection coverage, HTTPS/external origin flags, cookie attributes, script integrity/module/async/defer flags, resource outcomes, policy flags and category indicators. The v3 addition represents whether a limited enforced CSP source matcher excludes the target script. It is not an assertion that a script executed or was harmless.

Features use bounded ternary values: 1 for true, 0 for false and -1 for missing/unknown. Unsupported expressions and ambiguous matching stay unknown. Identifiers support application-side matching but are excluded from feature identities; raw URLs, names and training text are not supplied to the model. The target's information is preserved when surrounding context is capped. Capped or incomplete coverage can force review.

The sequence represents a structural snapshot, not runtime execution order. Header, cookie and resource context are not a time series of exploit behavior. A recurrent model can aggregate this ordering, but one should not defend the choice by claiming it learns temporal attacks. That would require a different collection protocol and evaluation.

PyTorch packs variable-length sequences so padding does not become observed context. JavaScript evaluates the actual bounded sequence. The logistic comparator flattens the same ordered information, pads unused steps with -1 and appends normalized length L/74, giving 74*46+1=3,405 input coordinates. StandardScaler fits means/scales on training data only and applies those values to validation/evaluation inputs. The LSTM directly receives the bounded ternary features.

4.3 LSTM equations and complexity

For step t, input x_t has dimension d=46 and hidden/cell states h_t and c_t have dimension h=8. With sigmoid denoted sigma and elementwise multiplication denoted *, the recurrence is:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

$$g_t = \tanh(W_g x_t + U_g h_{t-1} + b_g)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

$$h_t = o_t \odot \tanh(c_t)$$

$$z = w_{out}^T h_L + b_{out}, \quad p = \sigma(z/T)$$

The export uses PyTorch gate order ifgo. PyTorch stores input and hidden bias vectors separately; the runtime must reproduce their sum. The model has 4h(d+h)+8h+(h+1)=1,801 parameters under this architecture. A maximum-length recurrent pass requires approximately 74*4*8*(46+8)=127,872 multiply-accumulates before activation/output overhead. This is a calculated complexity estimate, not a measured latency guarantee. Numeric model JSON is capped at 128 KiB. [R1]

4.4 Training objective and optimization

For sample logit z_i, binary target y_i and mean-one weight w_i, the stable weighted binary cross-entropy objective is:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} w_i [\log(1 + \exp(z_i)) - y_i z_i]$$

The implementation uses BCEWithLogitsLoss for numerical stability, Adam with learning rate 0.01, gradient-norm clipping at 1, a registered random seed of 42 and one CPU thread. The published candidate experiment used 60 epochs and selected epoch 60 by balanced validation loss. Training supported JS-002 only: 96 eligible training rows and 32 validation rows; recorded validation loss was 0.0245420821. The broader development snapshot contained 300 examples, with 7/2/2 family assignments, but those are not all eligible binary training rows.

Class/category weighting is explicit and fitting-only statistics are not estimated from the holdout. Checkpoint selection uses validation data. The existing experiment did not run k-fold cross-validation or a large neural architecture search. A single registered protocol is

easier to audit but provides limited evidence about optimization variance across random seeds.

4.5 Calibration and abstention

Temperature T is selected on validation logits from {0.25, 0.5, 0.75, 1, 1.5, 2, 4, 8} using negative log likelihood. Candidate contextual thresholds are {0.05, 0.1, 0.2, 0.3, 0.4}; change thresholds are {0.6, 0.7, 0.8, 0.9, 0.95}. The selection prefers coverage subject to conservative validation precision/recall and no actionable example de-emphasized as contextual.

The active JS-002 policy uses T=0.25, contextual threshold 0.40 and change threshold 0.95. Thus p<=0.40 proposes contextual, p>=0.95 proposes change warranted, and the interval between them proposes needs review. Missing, unsupported or conflicting evidence can override any score to review. High/critical findings remain visible through aggregation safeguards.

The logistic comparator is a StandardScaler plus LogisticRegression model, trained on the same input information and family partitions with random_state=42 and max_iter=2000. Its recorded temperature is 4 and thresholds are 0 and 1, which result in complete abstention on the external holdout. This is an informative failure of that frozen decision policy, not a fair claim that every tuned logistic model is inferior to an LSTM.

4.6 New seven category pipeline

The new evidence-v4 project pipeline is distinct from the active JS-002 artifact and the historical trainer experiment named v4. The shared terminology must not be confused: the currently approved model still consumes evidence-v3; the new seven-category companion candidate has not been fitted.

The approved development target is 12 independent Node/Express application families, frozen as eight training/four validation families. Each category requires at least 30 distinct examples per class in training and 15 per class in validation, with each class present in at least three families. Forks, templates, reused security implementations, identifier-only changes and repeated resources cannot manufacture counts.

The active development snapshot is dataset 056, with 328 authenticated captures, 1,011 finding observations, and 326 eligible examples against the 630-example target. These are different units. A signed capture may contain several findings, and only admissible, materially distinct examples within each class and split count toward the target. The 12 registered families retain their 8/4 split. HomeInventory occupies a training slot and Etherpad a validation slot. Registered Etherpad HSTS captures observed during the interrupted follow-up are outside dataset 056 and its totals.

Category	Eligible of 90	Remaining	What the category must distinguish
HDR-001 HSTS	84	6	A warning on a header versus the browser's effective HTTPS upgrade behavior

Category	Eligible of 90	Remaining	What the category must distinguish
HDR-002 Framing	32	58	A sensitive action exposed to frames versus an intended embedding flow
HDR-003 Script policy	42	48	An enforceable restrictive policy versus one that permits an unapproved probe
CK-001 Secure	53	37	A cookie required for authentication versus a verified preference
CK-002 HttpOnly	61	29	Unnecessary script access versus verified native JavaScript dependency
CK-003 SameSite	32	58	Unnecessary cross-site use versus a needed, tested and protected flow
NET-001 Mixed resources	22	68	An HTTP dependency versus a verified pre-transfer HTTPS upgrade that preserves functionality
Total	326 of 630	304	22 class or family deficits at preflight

HSTS meets both training quotas at 30 examples per class across eight families. Validation has 12 of 15 per class and needs three more of each. Passing the HSTS development quota would establish data readiness for that category. Model fitting, unseen application evaluation and release checks are separate gates. No real seven-category candidate has been fitted and no new qualification holdout has been consumed.

Strict native HSTS collection now works in the recorded Ubuntu/Chromium loopback fixture with disposable certificate trust. The browser controls can verify plaintext before installation, a TLS attempt while a policy is active and plaintext after a verified reset. Resource controls compare actual transfer behavior and a native interaction with and without the dependency. These developer fixtures do not certify arbitrary live sites or the Windows certificate setup. A failed, interrupted or inadmissible run earns no credit.

After a real candidate is frozen, qualification requires a separate fresh application holdout. Collection, fitting and evaluation are developer-owned. No user annotations, private-account recruitment or background learning from user scans is part of the normal workflow.

5 Empirical Proof of ML Performance and Model Validation

5.1 Experimental protocol and leakage control

The strongest recorded evidence is the JS-002 external holdout evaluated on 26 October 2026. Twelve new MIT-licensed source families completed 216 browser scenarios: a baseline plus 17 script/CSP profiles per application. Meaningful original-application interactions and independent script-execution/CSP-blocking oracles ran with external traffic blocked. These were local imposed configurations; they are not claims that upstream applications contain those vulnerabilities.

The exposure registry contained eleven historical datasets, nineteen family identities and 307 numeric variants. Seventeen prior-input rows were rejected. The retained evaluation comprised eleven eligible families, 175 family/context rows and 159 distinct numeric contexts: 80 change-warranted and 79 contextual. Both classes occurred across all eleven families. No retained conflicting or uncertain rows were reported. The difference between 175 rows and 159 metric examples matters: repeated numeric contexts do not inflate the primary metric denominator.

Family-based partitions and canonical identity/grouping prevent variants of the same source family from straddling development and evaluation. Equivalent input grouping additionally detects leakage beyond repository names. The existing private family-identification key is preserved; opaque IDs are not reset to make old applications appear new. Source revisions, licenses, hashes, collection harness, candidate, category policy and executed JavaScript parity are bound to the registration.

An exclusive consumption marker is written before prediction. Interrupted/failed evaluations remain consumed. Checkpoints, calibration and thresholds are frozen before the external holdout; source replacement or threshold tuning after seeing test scores is prohibited. The scheme is a preregistered family holdout, not k-fold cross-validation. A future grouped/nested cross-validation study could measure development stability, but must not recycle this consumed external holdout into a fresh claim.

5.2 Quantitative comparison

System on the 159-context holdout	Precision	Recall	Non-abstention coverage	F1
LSTM model only	100%	100%	100%	1.000 calculated
LSTM after safeguards	100%	100%	100%	1.000 calculated
Frozen logistic policy	Undefined	0%	0%	Not reported due to complete abstention
Rule-only all-observations-change baseline	50.31%	100%	100%	0.6695 calculated

The rule baseline marks every observed JS-002 target as change warranted. Its precision is 80/159. It is a triage baseline, not a claim that the raw rule has a 49.69% proven-vulnerability false-positive rate. No exploitability ground truth is being evaluated.

The LSTM Wilson 95% precision and recall intervals are 95.42%-100%; coverage is 97.64%-100%. Paired family bootstrap intervals are retained in the aggregate JSON. Perfect observed outcomes produce degenerate family-bootstrap results; the small number of families and shared scenario machinery limit effective independence. These intervals do not justify universal accuracy or zero future false negatives.

No high/critical actionable contextual decision or insufficient-evidence contextual decision was observed. However, the holdout had no uncertain examples and JS-002 findings are ordinarily informational. High-severity and missing-context safeguards were tested separately as engineering fixtures; zero errors on absent subgroups is not a measured sensitivity estimate for those groups.

5.3 Confusion matrices and definitions

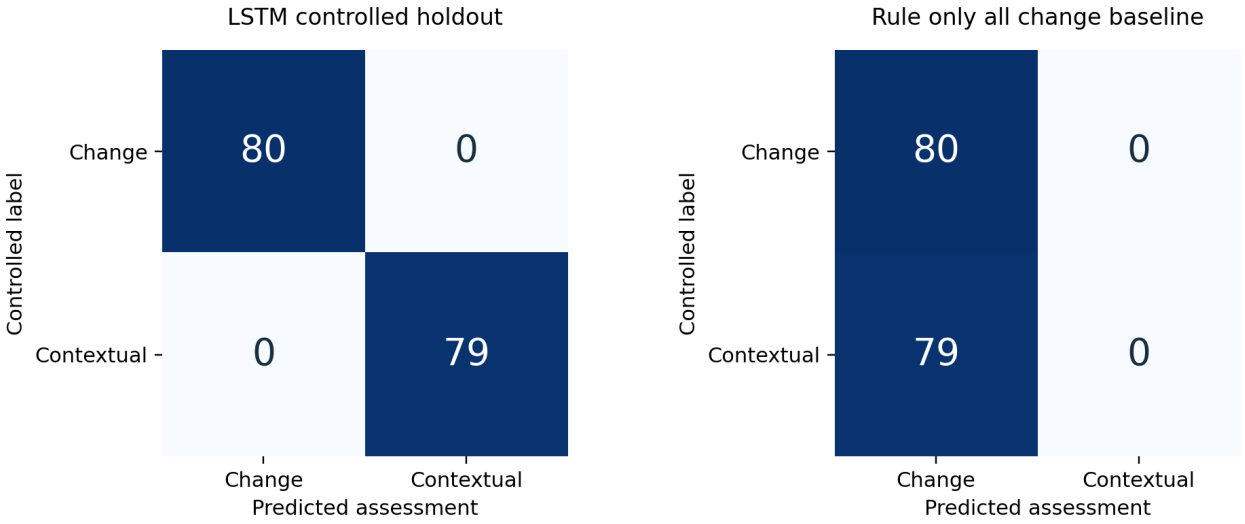


Figure 1 Derived confusion matrices from 159 unique controlled JS-002 contexts. Source E7. The frozen logistic policy abstained on every example and is not represented as a binary classifier here.

Positive means change warranted; negative means contextual. These matrices are CALCULATED from the recorded class counts and perfect precision/recall/coverage, rather than a separately published raw prediction CSV. The PDF uses annotated plots; the editable source retains the equivalent numeric tables.

The logistic policy places all 80 positive and 79 negative examples in Needs review. Abstention must remain explicit rather than being silently counted as a correct negative.

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN + \text{abstained positives}}$$

$$F_1 = \frac{2TP}{2TP + FP + FN} \quad \text{(fully decided binary case)}$$

$$\text{Coverage} = \frac{N_{\text{change}} + N_{\text{contextual}}}{N}$$

ROC-AUC, PR-AUC, calibration error and Brier score are not published for this release in the inspected aggregate record. They should not be invented or substituted with thresholded accuracy. RMSE and MAE are not the primary metrics for this classification task. A regression residual distribution is not applicable; probability-calibration residuals would be a separate unmeasured analysis.

5.4 Interpretability and error analysis

VYLNT explains observed facts, missing context, rule identity, matched resource context, assessment category and remediation references. This is evidence traceability and policy explanation. It is not SHAP, integrated gradients, causal attribution of LSTM weights or proof that a learned feature caused the prediction. No such attribution study or feature-importance ranking has been measured.

Zero observed errors on the retained holdout prevents a rich empirical taxonomy of model mistakes in that set. Development evidence is more revealing: twenty cookie-purpose counterexample pairs had identical observable inputs but opposite verified purposes. The category-v4 development record retained these conflicts rather than forcing a label. That demonstrates an information limitation: no deterministic or learned classifier can distinguish opposing labels from identical x without additional evidence.

The active v3 CSP matcher supports a narrow subset of source-list behavior and intersects multiple enforced policies. Nonces, dynamic trust, unsupported source expressions, redirects and contradictory/missing evidence require review. Matching only sanitized paths can be ambiguous when original queries differ; the application must retain request identity transiently while keeping stored evidence private.

The evaluation is vulnerable to controlled-suite regularity: many applications share imposed scenario profiles, the model may largely learn the observed CSP exclusion relationship, and real-world authentication/embedding intent is absent. A direct deterministic CSP relevance comparator is a worthwhile missing ablation. The successful LSTM release should therefore be defended as a scoped engineering experiment, not evidence that a neural network is necessary for all eight checks.

5.5 Release integrity and numerical parity

Python-to-JavaScript parity compared four numerical examples and 84 category/comparator decisions before evaluation. The maximum numeric error was 4.53e-8 against a 1e-6 tolerance. This supports export/runtime equivalence under those fixtures; it is not an exhaustive floating-point proof for all inputs.

The qualified model hash is 6fa227ae6f0e5c7259d3636066be05b0da5c661726c6acf77b3771178f105a1f. The candidate hash is f2ea93f213b51b2e1066f4fe910ed55052268e2533c5ce51d3ff833d38e7012e. Runtime acceptance records Worker/API agreement, automatic save, 71-observation grouping, reload/idempotency, JSON/PDF exports, archived verification, raw-report behavior, high-severity/missing-context handling, altered-model fallback, accessibility and narrow layout. These are historical recorded checks, not newly executed documentation benchmarks.

Release gates require precision at least 90%, recall at least 95%, coverage at least 50%, adequate fresh families/classes, zero prohibited contextual de-emphasis and improvement over the rule comparator with recall loss at most five percentage points. Categories are approved separately. A failing category remains experimental even when its code paths compile and unit tests pass.

6 Deployment Scalability and Production Readiness

6.1 Current serving model

The application runs as a local Node REST API with a React dashboard. Inference occurs in a browser Web Worker, with shared JavaScript analysis on the authenticated API for finalization and trusted verification. It is not a GPU service, gRPC endpoint or external AI call. Runtime analysis sends no scan to a third-party inference provider.

Supabase supplies hosted identity/database services. Training and Hardhat run separately. The built API serves the dashboard; Vite is optional for development. Follow the documented chain-node/deployment/API startup sequence and load the extension unpacked in Edge/Chromium. Ordinary scans need no developer TLS/corpus commands.

Configuration supports explicit origins/proxy trust, Supabase and optional shared Redis rate limiting. Defaults per 60 seconds are 600 requests/IP, 300/authenticated user and 60 writes, subject to route/middleware configuration. These are protection limits, not measured throughput. Production Redis was unavailable in the earlier setup; multi-instance limiting remains unverified.

6.2 Recorded latency and resource evidence

Workload	Chromium	Edge	Interpretation
Typical worker report, 318 findings / 196,705 bytes	154 ms	250 ms	One local sample per runtime
Accepted near-limit report, 523,855 bytes	112 ms	125 ms	Metadata size stress, not more unique contexts
Maximum observed main-thread timer gap	11.3 ms	12.2 ms	One measurement; UI responsiveness signal

These 27 October worker measurements used Chromium 156.0.8078.4 and Edge 156.0.4314.42 on the recorded Windows machine. Near-limit execution being faster than the typical sample is not contradictory: byte size is not unique-context count, and machine scheduling varies. No P50/P95/P99 inference distribution or sustained concurrency result is available.

Historical 20 October raw-renderer measurements on an Intel Core i5-9300H, eight logical CPUs, approximately 15.8 GiB RAM and Node 20.18.0 used three runs per profile. Typical PDF rendering ranged 1.757-2.364 seconds; a 153-finding long-URL stress case ranged 6.058-8.242 seconds with sampled Node-plus-Chromium working-set peaks of 461.4-473.2 MiB. These predate later PDF tagging/finalization and do not benchmark the current full authenticated insights pipeline. The stress case also exceeded accepted stored-report size. They are historical instrumentation evidence, not capacity guarantees.

Local PGlite storage tests recorded first-history-page medians of 4.07 ms at 1,000 scans and 7.84 ms at 10,000 scans across 100/1,000 sites. The corresponding 10,000-scan comparison median reached 124.66 ms. A 1,000-scan file store recorded a 40.05 ms review-write median. These exclude network/authentication/seeding, use warm local workloads and do not represent hosted Supabase latency. Complete methodology is in docs/PERFORMANCE.md.

6.3 Bottlenecks and scale strategy

The PDF renderer currently serializes work per API process and rejects overlapping generation as busy; Chromium launch and memory are material costs. The anchor worker is also serialized to control nonce allocation. The local file adapter reads/rewrites a whole file and is not multi-instance storage. An offset-paginated history and exact comparison queries can become increasingly expensive with growing history.

A proposed production path is PostgreSQL-backed ownership/history, measured indexes and cursor pagination, shared rate limiting, a bounded PDF job service with per-owner quotas, durable queue ownership and a single accountable blockchain writer per deployment. Stateless API replicas alone would not solve local-journal synchronization or signer nonce coordination. These changes are proposed, not deployed.

A proposed production study should time collection, inference, authentication, storage, finalization and export separately, with 100 measured warm/cold trials per workload at concurrency 1/2/4. Record hardware/browser/build, timeouts, memory and P50/P95/P99. Existing single worker samples and three historical PDF renders cannot establish these percentiles.

6.4 Security and operational readiness

The API validates input, restricts origins, rate-limits requests and checks ownership; RLS adds database enforcement. Operations should use the caller's scoped identity. Supabase publishable configuration is public, while secret/service-role keys must stay outside the extension, dashboard and reports.

Host/path, time, findings and transaction correlations remain sensitive metadata. JWTs, signing keys, private annotations, synthetic credentials and database contents stay outside report/model artifacts. Public hosting needs a review of the separate demo-ledger endpoint: anonymous transaction grouping still discloses metadata.

Companion builds execute npm scripts with the OS user's privileges. Temporary checkout/data cleanup and loopback restrictions reduce accidental exposure; they do not contain malicious dependencies or scripts. Public multi-tenant execution requires stronger isolation and credential controls.

Migration 006 is user-reported applied, with independent hosted acceptance still pending in the project plan. Archived envelopes, owner tags and size boundaries have local tests. Recorded strict native TLS experiments use disposable Ubuntu trust. The normal extension/dashboard workflow does not depend on that developer test setup.

7 Judge and Interview Defense Cheat Sheet

7.1 Why use an LSTM when several checks are deterministic

Answer: the raw checks are deterministic and remain shared pure functions. The learned task is relevance assessment over bounded evidence context. The small LSTM is one tested architecture, with a logistic comparator and rule-only baseline. We have not proven that recurrence is necessary; the JS-002 relationship may be largely explained by a narrow CSP exclusion feature. A direct policy-only relevance baseline and grouped ablation study are the next defensible comparisons. Do not claim temporal attack recognition from a structural snapshot.

Mathematical anchor: d=46, h=8, L<=74 yields 1,801 parameters and bounded per-variant computation. The output is an abstaining classifier, not a vulnerability probability.

7.2 How credible is the reported perfect ML performance

Answer: 100% precision/recall/coverage was measured on 159 deduplicated controlled JS-002 contexts from eleven eligible fresh families. The protocol froze the candidate and wrote an exclusive consumption marker before inference. Precision and recall Wilson lower bounds are approximately 95.42%. The suite has shared imposed profiles, no uncertain test rows and no high/critical subgroup coverage; it does not establish universal accuracy. The logistic policy completely abstained, so the result cannot establish broad LSTM superiority.

Mathematical anchor: TP=80, TN=79, FP=FN=0 for that retained fully decided set; the rule-only precision is 80/159. Family dependence limits naive independent-sample interpretations.

7.3 What does blockchain prove and what happens after tampering

Answer: compute the canonical frozen JSON digest again and compare it with an authenticated, live-verified local anchor. Changing a substantive field changes the digest with overwhelming collision resistance; modifying both a file and its own hash does not modify the trusted original anchor. Verification also checks source linkage, instance, bytecode, receipt/event/block and contract lookup. The chain proves content identity under its trust assumptions, not finding correctness. A locally controlled chain can be reset and is not public independent attestation.

Mathematical anchor: d=SHA256(canonical(snapshot)); compare recomputed d with the contract record. The PDF file's byte hash is a different object and is not implicitly anchored.

7.4 Why does GitHub passing not prove that the submitted code is fixed

Answer: there are two workflows. The uploaded-report demo evaluates the report's observations and lacks fresh commit binding. The fresh gate actually checks out, builds and scans the selected commit, failing closed on execution/coverage errors. Even that covers sampled pages and eight rules, not all application security. A local hook is bypassable, and authoritative merge prevention requires repository protection that the owner controls.

Mathematical anchor: block iff there exists a finding with severity at least high and confidence high; ML scores and review notes do not alter this predicate. A zero-blocker result is conditional on the scan's coverage and execution success.

7.5 What is the largest architectural risk and what would you change first

Answer: the largest current scientific risk is insufficient native evidence for the remaining categories; identical inputs with opposite purposes cannot be fixed by more epochs. The operational risks are trusted project execution, browser/PDF cost and local journal/signer state. First complete trustworthy category data, then measure fair baselines and deployment workloads. Public multi-tenant execution needs a proper sandbox, shared durable orchestration and separately managed secrets before horizontal scaling.

Mathematical anchor: if x is identical for two examples with opposing labels, a deterministic decision f(x) cannot correctly assign both. Add distinguishing evidence or abstain. Increasing hidden size does not solve missing information.

Appendix A Presentation and Live Demo Guide

A.1 Ten minute presentation sequence

Time	Suggested slide	Speaker objective
0:00-0:45	Problem and scope	Configuration warnings need context; define the eight-check boundary
0:45-1:45	System diagram	Explain extension, API, persistence, worker, snapshot and local chain
1:45-2:45	One scan and grouped findings	Show evidence and coverage; preserve raw severity
2:45-4:00	Model and abstention	Explain 46-feature bounded input, LSTM, unknowns and thresholds
4:00-5:00	Actual evaluation	Present 159-context result, rule comparator and limitations together
5:00-6:30	Frozen report and tamper check	Export JSON, verify original, change a copy and show mismatch
6:30-7:30	Local ledger	Show real transaction metadata and explain disposable-chain scope
7:30-8:30	GitHub workflow	State which workflow is shown and what its pass/fail establishes
8:30-9:15	Local-project extension	Explain exact-commit provenance and unfinished native model work
9:15-10:00	Engineering trade-offs	Close with measured scope, missing evidence and next qualification work

A five-minute version uses the problem, architecture, grouped scan, evaluation and tamper demo. Keep GitHub/local-project details for questions. This is a slide-by-slide presentation guide, not a separately generated slide deck.

A.2 Demo preparation checklist

Use only an owned/local controlled website and disposable accounts. Follow RUN_COMMANDS.md for build/browser installation, chain node, contract deployment and API startup. Keep the local node running; after a reset, deploy again and restart the API so its deployment identity is current. Check dashboard sign-in, extension host permission and collection coverage before judges arrive. Confirm which storage mode is active without displaying configuration secrets.

Prepare an original frozen insights JSON and a separately edited copy. Use the same source scan and avoid altering the original demonstration artifact. Verify the API and chain are available before attempting a live match. Keep a recorded successful acceptance result as a fallback, explicitly labelled recorded if the live environment fails. Never substitute a cached confirmed badge for a live verification claim.

For GitHub, decide in advance whether the demo is the exported-report check or fresh-commit gate. Use a feature branch and controlled repository. If branch protection is absent, show the failed check rather than claiming GitHub prevented the branch push or merge. For the report workflow, fix the website, rescan and commit the new report; explain that freshness/commit binding is not enforced by that workflow.

A.3 Live demonstration script

Say: "This is a development configuration scan. The extension can observe these headers, scripts and cookie attributes; it cannot prove exploitability." Scan and point out the coverage indicators. Open the dashboard and compare one raw finding with its grouped insight. If dozens of equivalent scripts exist, show the affected count and retained evidence references.

Say: "Only script-integrity relevance is currently qualified within our controlled scope. Other categories remain conservative." Open the insights and show facts, missing context and conditional remediation. Show the separate raw PDF to demonstrate that grouping has not deleted evidence or changed severity.

Say: "The API freezes the original scan and analysis, then hashes canonical JSON." Show the saved version digest. Open anchor status and the ledger transaction. Upload the original JSON to verification, then the edited copy. Explain content mismatch separately from unavailable-chain or unanchored results. A substantive change should fail original identity; whitespace-only reformatting can retain canonical identity.

For GitHub, show a controlled blocking finding and then a new passing report/fixed-commit run. Explain the selected blocking predicate and the difference between warnings and blockers. Do not describe all medium findings as fixed merely because a check passes.

A.4 Short spoken answers

What does the model do? "It assesses relevance for a supported rule context; it does not discover arbitrary vulnerabilities or write fixes."

Why two PDFs? "One preserves the complete raw observations. The other presents grouped assessments and reviewed guidance from the frozen insight version."

Is this a real blockchain? "Yes, it executes real EVM transactions and contract state changes, using a disposable local Hardhat chain with one authorized writer. It is not independent public attestation."

Why more data? "The existing twelve-family holdout qualified JS-002. Cookie purpose, CSRF and other categories need different evidence; the old script scenarios do not answer those questions."

Why local-project registration? "It authorizes one repository and binds scans to committed code. The browser dashboard cannot freely execute arbitrary local projects."

A.5 Failure handling during judging

If scanning is disabled, inspect tab type and granted site access, then use the prepared localhost site. If saving fails, distinguish authentication/storage failure from scan success. If the chain is unavailable, keep the report visible and state verification unavailable. If its instance changed, redeploy/restart rather than reusing old receipts. If PDF generation is busy, retry after the active render rather than promising a queue that does not exist.

If a novel website yields Needs review, show the missing context and the original finding together. If a developer TLS probe fails, retain the failed outcome and describe the fixture requirements. Use the recorded Ubuntu runs when explaining strict HSTS acquisition.

Closed-book rehearsal. Draw the extension, shared rules, API, database, worker, frozen report and chain in 60 seconds. Then trace one scan ID, both content digests and the transaction hash. Explain one true observation, one uncertain assessment and what the ML holdout established. Finish by naming the CI workflow you demonstrated and what its pass establishes.

Appendix B Evidence Register and References

B.1 Repository evidence register

ID	Repository evidence	Claims supported
E1	PLAN.md and docs/ML_PROJECT_QUALIFICATION.md	Current status, open categories, recorded increments
E2	apps/extension/manifest.json and packages/core/index.ts	Permissions, shared checks and deterministic findings
E3	packages/core/evidence-context.ts and evidence-v3.ts	Numeric inputs, matching, unknowns and CSP constraints
E4	packages/ml/controlled-category-release.ts and bundled-evidence.ts	Actual active release, policy and approved JS-002 scope
E5	ml/insights.py, insights_v3.py and insights_v4.py	Historical candidate training, calibration and logistic comparator
E6	ml/evaluations/category-v4-2026-10-26.json	Candidate training counts, selection and cookie conflicts
E7	ml/evaluations/fresh-holdout-js002-2026-10-26.json and .md	Fresh controlled qualification counts, metrics and intervals
E8	ml/evaluations/fresh-holdout-js002-runtime-2026-10-26.json	Recorded Worker/API/export/reload and safeguard acceptance
E9	packages/ml/frozen-insights.ts and supabase/migrations	Immutable envelopes, limits, archived verification and ownership
E10	contracts/VyIntReportAnchor.sol and apps/server/anchor.ts	Contract semantics, durable jobs and live verification
E11	packages/core/gate.ts, .github/workflows/security-gate.yml and docs/M4_SECURITY_GATE.md	Exact-commit policy, workflow and exit semantics
E12	docs/REPORT_VERIFICATION.md and dashboard workflow generator	Uploaded-report boundaries and tamper workflow
E13	docs/PERFORMANCE.md	Dated worker/PDF/storage measurements and sampling caveats
E14	docs/ML_REGISTERED_DATASETS.md and private dataset 056 audit	Active split, material example counts and remaining class gaps
E15	docs/ML_TLS_EVIDENCE.md and docs/ML_RESOURCE_COMPATIBILITY.md	Browser transport controls, strict fixture scope and remaining evidence limits

ID	Repository evidence	Claims supported
E16	RUN_COMMANDS.md and package.json	Reproducible entry points and dependency declarations
E17	apps/server/app.ts and docs/ML_PROJECT_FROZEN_REPORTS.md	Route contracts, upload outcomes and project provenance

Repository references are relative to the VYLNT checkout. Private keys, account tokens, annotations and synthetic credentials are intentionally not reproduced. Model and candidate hashes are public artifact identities, not access credentials. The facts in this document should be revised when implementation or evaluation evidence changes; a publication date alone does not certify new code.

B.2 External technical references

[R1] PyTorch 2.7 LSTM API and recurrent equations: https://docs.pytorch.org/docs/2.7/generated/torch.nn.LSTM.html

[R2] Supabase PostgreSQL row-level security: https://supabase.com/docs/guides/database/postgres/row-level-security

[R3] Chrome extension service-worker lifecycle: https://developer.chrome.com/docs/extensions/develop/concepts/service-workers/lifecycle

[R4] Hardhat 3 Node.js support: https://hardhat.org/docs/reference/nodejs-support

[R5] Playwright request lifecycle and redirects: https://playwright.dev/docs/api/class-request

[R6] CSP upgrade-insecure-requests semantics: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Content-Security-Policy/upgrade-insecure-requests

[R7] HSTS semantics and host applicability: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Strict-Transport-Security

[R8] Git hooks and pre-push behavior: https://git-scm.com/docs/githooks

[R9] GitHub protected-branch semantics: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches

External references explain platform semantics; they do not establish VYLNT performance. Exact project measurements come from E6-E8 and E13. Architecture rationales in Section 2 are engineering interpretations of the implementation unless a measured comparison is explicitly identified.

Appendix C Follow One Report From Browser to GitHub

C.1 The one-minute explanation

VYLNT helps website developers review security-relevant browser configuration. An Edge extension observes a permitted page and applies eight shared rules. An authenticated API saves the scan, then a dashboard worker groups findings and adds evidence-backed assessments. A small LSTM can help assess script-integrity observations within its qualified scope. The API rechecks and freezes the insights. The saved JSON can be verified against its original digest, and the local blockchain can independently supply the recorded digest for that deployment. GitHub checks help a team catch blocking observations in a fresh commit scan or a separately uploaded report.

C.2 Choose the right entry point

When you want to know	Use	Evidence you get
What the browser can see on a page you are using	Edge extension	Observations from that tab and time, with permission and collection status
What a committed local website revision exposes on specified routes	Dashboard Local projects	A new scan from an isolated Git checkout and signed project provenance
Whether a code change introduces a blocking observation	Fresh-commit gate or pre-push hook	A fresh browser scan of the selected commit with a pass, block or error
Whether a saved report file matches your account's reference	Verify report	A database content match or mismatch, with a live anchor result for supported insights JSON
Whether a committed JSON report contains a blocking observation	Dashboard-generated report workflow	A policy decision on that submitted report

The ordinary workflow starts with the extension. The companion is optional and answers a different question because its source is an exact commit. The CI gate runs independently from the dashboard, Supabase and blockchain.

C.3 Run the demo and trace the identifiers

On a fresh checkout, use Node 24 and run `npm.cmd ci --include=optional` and `npm.cmd run setup:browser`. Prepare the private environment configuration before starting the API. For the configured hosted-storage and local-chain demo, the public settings are `VYLNT_STORAGE=supabase`, `VYLNT_BLOCKCHAIN=local`, `HOST=127.0.0.1` and `PORT=4318`. Use the Supabase setup in `docs/API.md` for the project URL and publishable key. Keep credentials out of demo screenshots.

1. On a configured checkout, start the chain in the first terminal with `npm.cmd run chain:node`. Keep it open. In a second terminal run `npm.cmd run build`, `npm.cmd run chain:compile`, `npm.cmd run chain:deploy` and `npm.cmd run dev` in order. The dashboard is at `http://127.0.0.1:4318`. See E16 for first-time dependency, browser and environment setup.
2. In Edge, reload the unpacked extension from `dist/extension`. Open a controlled website you own, open the side panel and grant access to that site if required. Connect the extension using the same account as the dashboard. Scan or rescan and check the three coverage indicators. Use a fixture with a known observation so you can explain exactly what the rule saw.
3. Open the saved scan in the dashboard. The `scanId` identifies the original record. The original scan digest identifies that record's canonical contents. The website list groups scans for navigation. A domain label alone is insufficient to identify a particular scan.
4. Open a raw finding and point to its rule ID, severity, confidence and evidence. Then open the corresponding insight group. Explain how the group can cover multiple observations while retaining their finding IDs. Show the evidence and verification instructions. Treat the configuration score as a prioritization aid. The score uses the largest severity weight per rule, so duplicate instances do not repeatedly deduct the same rule weight.
5. Wait for the dashboard to finish saving the insights version. The frozen insights digest identifies the combined snapshot, including the original scan linkage and analysis identity. It differs in purpose from the original scan digest. Download the raw PDF for the audit trail and the insights PDF for the prioritized narrative. Download insights JSON for the tampering demonstration.
6. In the separate ledger at `http://127.0.0.1:4318/ledger.html`, locate the digest and show the transaction hash and block. The report digest identifies content. The transaction hash identifies the transaction that recorded it. The deployment instance tells you which local chain history the proof belongs to.
7. Open **Verify report** and upload the original insights JSON. Then alter a substantive field inside `content` in a separate copy, keeping its original report ID, and upload that copy. Describe the two results. Whitespace and object-key ordering can leave canonical identity intact. Changing a covered field changes the recomputed digest.
8. For CI, demonstrate one workflow at a time. In the fresh gate, commit a controlled fix and show the new commit being built and scanned. In uploaded-report mode, fix the site, rescan it, export fresh JSON and commit it at the configured path. Show the job summary for a blocker and then a pass. Explain any remaining warnings.

C.4 Read the pipeline as a sequence of contracts

```
Allowed tab          -> sanitized observation and coverage
Shared rules         -> finding IDs, severity, confidence, score
Authenticated API    -> owner-scoped scan and original digest
Dashboard worker     -> evidence contexts and grouped analysis
API finalization     -> frozen version with a canonical digest
Anchor journal       -> transaction attempt and receipt state
Live verification    -> content result and chain result
GitHub / local hook  -> independent policy check for its input
```

Interface	What passes across it	Why this boundary matters
`POST /api/scans`	Validated scan submission	The API uses authenticated ownership and validated observations
`GET /api/scans/:id`	One saved scan	History is separate from the current tab
`POST /api/scans/:id/insight-reports`	Finalization request with analysis identity	The API derives trusted frozen content before persistence
`GET /api/insight-reports/:id.json and .pdf`	Frozen export for that version	JSON and PDF share the saved source snapshot
`POST /api/reports/verify-upload`	Raw scan or supported insights JSON	Raw matching and live anchor verification have different outcomes
`GET /api/insight-reports/:id/anchor`	Anchor job metadata	Saved report availability and chain confirmation are separate
`GET /api/insight-reports/:id/verify`	Live owner-scoped verification	Rechecks the deployment and receipt instead of trusting cached status
`POST /api/projects/:id/runs`	Run request for a registered project	The local companion resolves a permitted commit and produces a new scan

C.5 Explain the visible results and recover during a demo

What you see	What it means	What to check
Collection unavailable for one evidence source	The browser could not observe that source	Grant the origin permission, confirm a supported HTTP(S) tab and rescan
Side-panel scan succeeds but dashboard history is empty	Collection and synchronization are separate	Check account identity, API availability and the extension's save status
Needs review	Evidence or compatibility context is missing or ambiguous	Open the issue and explain which fact would settle it
Raw JSON matches the database	Canonical original scan matches its owner-scoped reference	Upload frozen insights JSON if you want a live anchor check
Content mismatch	Recomputed uploaded content disagrees with the saved reference	Preserve the original and inspect the edited copy
Report saved with verification unavailable	The chain or trusted reference could not establish a live result	Check node, deployment instance and API configuration
Saved insights are unanchored	This version has no verified anchor in the current environment	Save or finalize the version while the configured local chain is available
GitHub exit 1	The workflow evaluated findings and found a blocker	Read the rule, affected page and recommended change
GitHub exit 2	The workflow could not complete its contract	Fix the build, readiness, coverage or input path before interpreting security

C.6 Find the implementation during a technical interview

Topic	Start here	Then follow
Browser collection	`apps/extension/background.ts` and `content.ts`	`sync.ts` and `packages/core/index.ts`
API boundaries	`apps/server/app.ts`	Storage adapters and `supabase/migrations`
Browser analysis	`apps/dashboard/src/insight-worker.ts`	`packages/ml/controlled-category-release.ts` and shared evidence types
Report identity	`packages/ml/frozen-insights.ts`	API verification routes and `docs/REPORT_VERIFICATION.md`
Blockchain	`contracts/VylntReportAnchor.sol`	`apps/server/anchor.ts` and the ledger UI
Exact-commit checks	`apps/cli/gate-runner.ts`	`packages/core/gate.ts` and `.github/workflows/security-gate.yml`
Local projects	`apps/cli/project.ts`	`apps/cli/project-scanner.ts` and project signing
ML qualification	`docs/ML_PROJECT_QUALIFICATION.md`	`docs/ML_REGISTERED_DATASETS.md` and the published JS-002 evaluation

Appendix D Closed-Book Interview Drill

D.1 Browser and security

Why does VYLNT need extension permissions? It needs scoped access to observe the selected page's policies, DOM, requests and cookie metadata. Additional origins require an explicit permission grant.

What does Observed mean? Collection succeeded for that source in the scan. The findings and coverage together define the result. They cannot establish full website security.

What does HSTS do? A valid policy received over HTTPS instructs supporting browsers to use HTTPS for subsequent requests to the applicable host. A redirect is a separate server behavior. The strict fixture observes actual plaintext and TLS attempts.

What is the difference between Secure, HttpOnly and SameSite? Secure constrains cookie delivery to secure transport under browser rules. HttpOnly limits access from page JavaScript. SameSite limits when the cookie is sent in cross-site contexts. Application intent and tested flows determine whether a specific configuration is appropriate.

How do CSP and SRI differ? CSP constrains what the browser may load or execute. Subresource Integrity checks whether fetched bytes match the declared digest. A script observation can need both delivery and policy context.

Why can a mixed-content reference still need review? A DOM reference can be blocked or upgraded before transfer. A successful transfer also leaves execution and functional dependence to be established separately.

D.2 Data and ML

Why represent unknown explicitly? Missing evidence and observed false are different. Collapsing them teaches the model conclusions the collector cannot support.

Why is there a sequence if the system is not observing execution order? The encoder organizes a target and bounded surrounding structure for the model. Structural order provides context. Runtime execution needs separately observed evidence.

Why group 71 scripts? Equivalent context can share inference and presentation. The grouped issue retains each finding reference and its affected count. This is deterministic behavior.

Where do the fixes come from? A reviewed versioned remediation catalog and observed facts supply narrative and verification steps. The LSTM supplies a bounded relevance decision.

Why use family splits? Related source variants can share behavior and code. Splitting by row can leak those relationships and overstate generalization.

What is the exact model claim? JS-002 reached 100% observed precision, recall and decision coverage on 159 retained controlled contexts across eleven eligible fresh families. The confidence intervals, scoped scenarios and absent subgroups constrain the claim. The other seven categories are unqualified.

Why are there 328 captures and 326 eligible examples? A capture is a signed browser run. Findings are rule observations. Eligible examples are countable cases after provenance, class, distinctness and quota checks. These units answer different questions.

What closes ML-Q? Complete the seven development datasets, train and freeze a candidate, evaluate each category once on a fresh holdout, pass safety and quality gates, and verify the qualified release across the worker, API and frozen reports.

D.3 Storage, blockchain and CI

How do you stop one account from opening another account's report? API routes derive ownership from the authenticated caller. Hosted database policies enforce ownership at storage. A supplied ID identifies a record without granting access.

Why recompute analysis on the server? A browser can be modified. The server derives the frozen report from the saved scan and trusted versioned policy before hashing and queuing it.

What survives deletion? The stored scan and linked reports can be removed through the application. A digest already written on the current chain remains there until that disposable chain history is reset.

Can someone edit the PDF and keep its verification badge? The anchored digest covers canonical frozen JSON. The verification feature compares supported JSON with its saved reference and optional live chain record. PDF-byte verification would need a separate signature or digest.

What does the contract know about users? It records digests and transaction facts. The demo page derives anonymous account grouping from the local API journal.

Why persist a signed transaction before broadcast? The network response can be lost after the transaction was accepted. Persisted signed bytes let the worker reconcile or rebroadcast the same transaction during recovery.

Can a GitHub Action prevent an ordinary branch push? Actions evaluate after the branch update. A local pre-push hook can block that local push. Preventing a failing merge requires a repository policy that requires the check.

Will medium findings fail the gate? Medium findings are warnings under the current policy. The gate blocks high or critical findings at high confidence and separately fails when the scan cannot complete.

Why does the demo website's secure/insecure switch matter? It changes what the site actually serves. The fresh gate judges the resulting browser findings. The uploaded-report workflow judges the findings inside the supplied JSON.

What would you measure before production? Per-stage latency and memory at known concurrency, browser coverage under failures, database ownership, signer recovery and isolated execution for untrusted applications. Use the recorded local samples as baselines for that study.